Proyecto

Proyecto AquaSenseCloud

Infraestructura para la
Computación de Altas Prestaciones
Curso: 2024/2025

Rubén José Tenza Martín
Fernando Piñera Hervas

ÍNDICE

Tarea 1..... 3
Tarea 2..... 4
Tarea 3..... 11

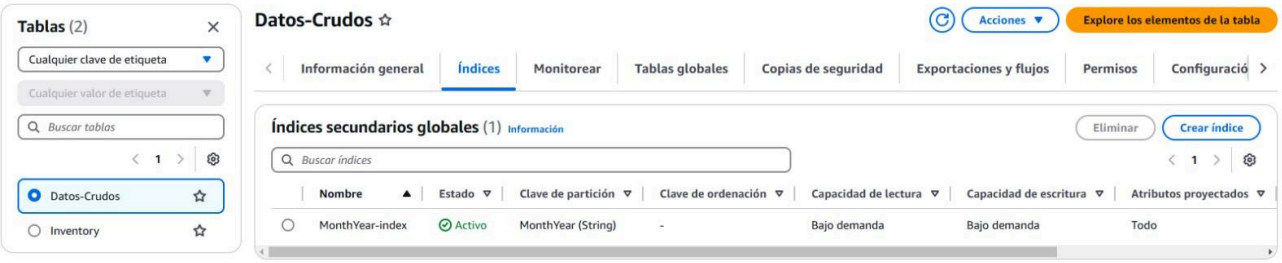
Tarea 1: Creación de las tablas DynamoDB

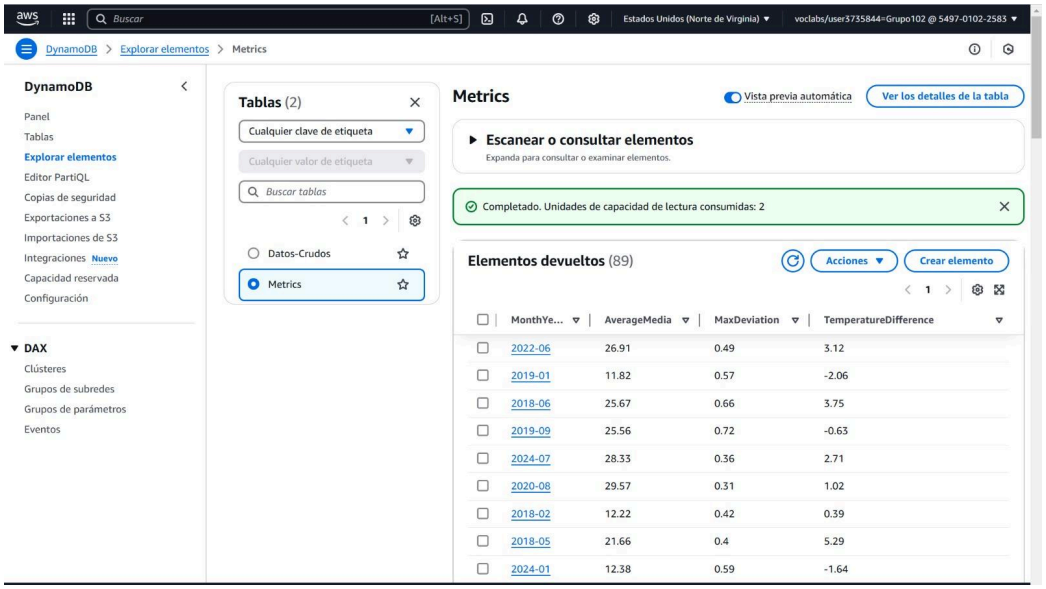
Primeramente creamos las tablas DynamoDB que serán las encargadas de almacenar los datos y posteriormente los datos procesados de los archivos .csv que iremos añadiendo al bucket S3.

Nos dirigimos al apartado de DynamoDB en AWS y ahí creamos la primera tabla, llamada Datos-Crudos, dicha tabla la pensamos con la intención de almacenar los datos del csv para no congestionar el proyecto y tener todos los datos mejor divididos. A dicha tabla le agregamos como clave de partición la columna Fecha y creamos un índice llamado MonthYear-index. Creemos que el índice nos ha ayudado en la mejora del rendimiento de consultas y permite un acceso más flexible a los datos, ya que sin el índice solamente podríamos acceder a la tabla a través de la clave primaria definida.

La segunda tabla que hemos creado se llama Metrics y es la encargada de almacenar los datos ordenados por mes y año, con el indice que creamos anteriormente. Esta tabla es de la que obtendremos los datos para realizar las consultas con el servidor web que crearemos y explicaremos

posteriormente.

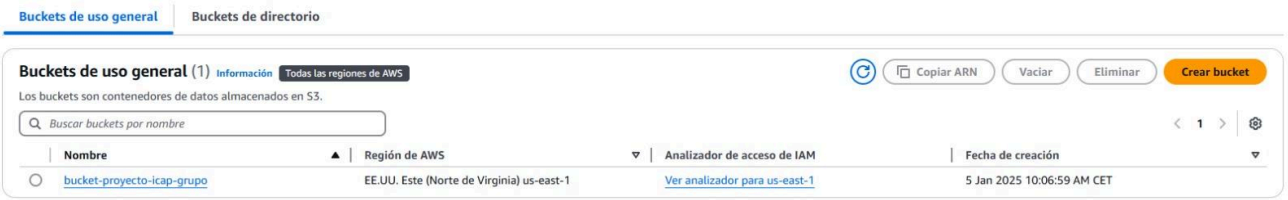




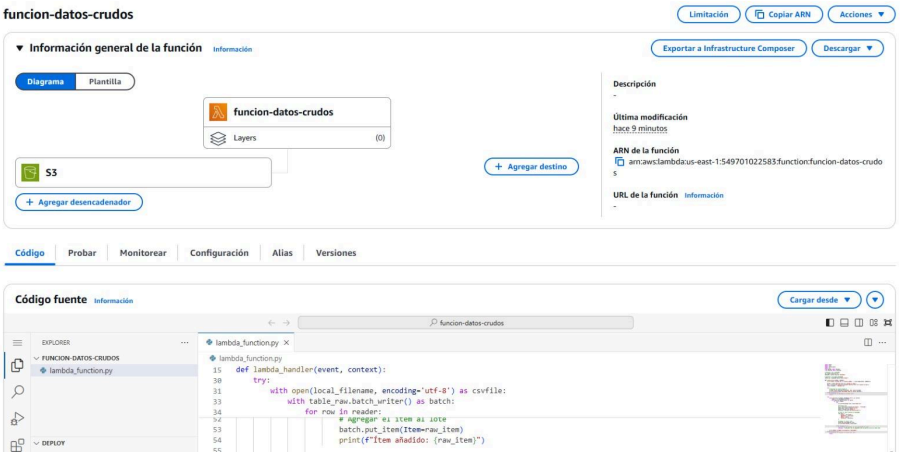
Tarea 2: Creación Bucket S3, funciones Lambda y notificación de eventos.

En esta tarea realizamos la creación del bucket S3 mediante el cual podemos almacenar y organizar datos en forma de objetos, son el componente principal del almacenamiento y sirven como espacio de almacenamiento en la nube para archivos de distintos tamaños.

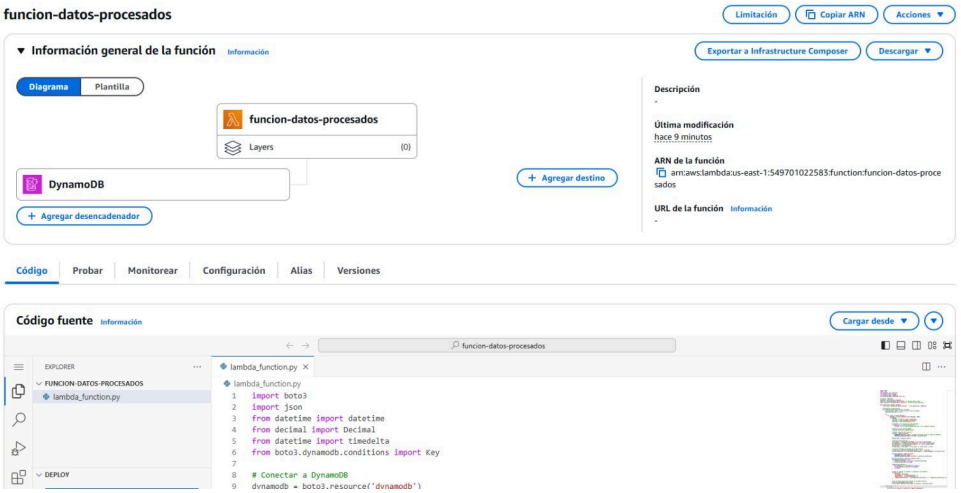
En este bucket será donde carguemos los archivos csv para que sean procesados por las funciones Lambda que vamos a crear a continuación. Crearemos 3 funciones Lambda.



La primera función lambda que vamos a crear es la llamada funcion-datos, con un entorno Python y con los permisos necesarios dados por el rol LabRole. El código de esta función hace que se active automáticamente cuando se sube un archivo al bucket S3, procesa dicho archivo CSV y carga los datos en una tabla DynamoDB llamada Datos-Crudos. Al activarse, la función extrae del evento recibido el nombre del bucket y la clave del archivo, descarga el archivo CSV a un directorio temporal (/tmp/data.csv), y luego lo procesa fila por fila utilizando un lector de CSV. Para cada fila, transforma los datos: convierte el campo Fecha al formato de mes y año (MonthYear), y convierte los campos numéricos (Medias y Desviaciones) a decimales. Cada fila se almacena como un ítem en la tabla DynamoDB utilizando batch_writer para inserciones eficientes. Si ocurre un error al procesar una fila específica, la función lo registra en los logs y continúa con la siguiente fila; en caso de fallos más críticos (como al descargar el archivo o procesar el CSV completo), se lanza una excepción y se detiene la ejecución.

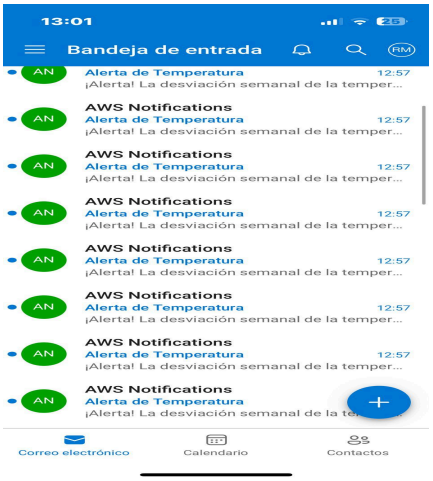


La segunda función Lambda es la llamada funcion-metricas, tiene un entorno Python y los permisos dados por LabRole. En esta función añadimos un desencadenador de la tabla Datos-Crudos para poder acceder a los datos de dicha tabla y realizar las operaciones necesarias para la inserción de los datos en la segunda tabla Metrics, su código implementa una función Lambda que se activa mediante un trigger de DynamoDB Streams cuando ocurren cambios en la tabla Datos-Crudos. Su propósito es procesar los datos mensuales y calcular métricas agregadas que se almacenan en otra tabla DynamoDB llamada Metrics. Para cada nuevo registro, la función identifica el mes y año (MonthYear) asociado y consulta todos los datos correspondientes a ese período usando un índice secundario. Calcula métricas como el promedio de medias (AverageMedia), la desviación máxima (MaxDeviation), la temperatura máxima del mes, y, si es posible, la diferencia de temperatura máxima respecto al mes anterior. Estas métricas son almacenadas en la tabla Metrics mediante un batch_writer. La función evita procesar duplicados verificando que cada mes-año sea manejado una sola vez por ejecución. Si ocurre algún error durante el procesamiento o consulta, este se registra y la ejecución lanza una excepción.



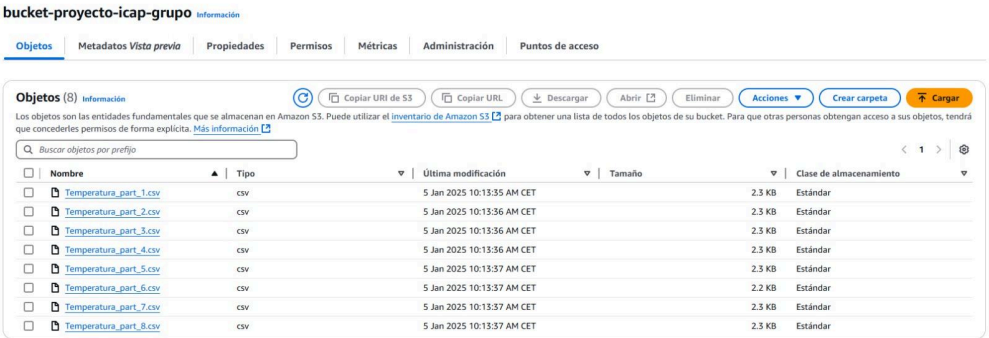
La tercera función es la llamada funcion-sns y es la encargada de la notificación por correo electrónico cuando un dato de la desviación semanal de la temperatura del agua supera el umbral establecido en 0.5. También tiene un entorno Python y los permisos de LabRole, para esta función primeramente debemos crear un tema en el servicio SNS (Simple Notification Service) y posteriormente crear una suscripción para poder establecer bien las notificación por la superación del umbral. En esta función añadimos un desencadenador de la tabla Datos-Crudos para poder acceder a los datos de dicha tabla. Su código se encarga de monitorear desviaciones anómalas en datos de temperatura registrados en DynamoDB y envía alertas mediante Amazon SNS si las desviaciones exceden un umbral de 0.5. La función procesa un evento de DynamoDB Streams, que contiene nuevos registros (NewImage). Para cada registro, extrae los valores de Deviation y Fecha, verifica si la desviación supera el umbral y, si es así, agrega un mensaje de alerta a una lista acumulativa. Al finalizar el procesamiento, si hay alertas, construye un mensaje consolidado con todas las anomalías detectadas y lo publica en un tópico SNS especificado por sns_topic_arn. La función registra errores o advertencias en los logs si faltan campos, no se pueden procesar valores, o si ocurre alguna otra excepción. Finalmente, devuelve una respuesta indicando la cantidad de registros procesados y las alertas detectadas.

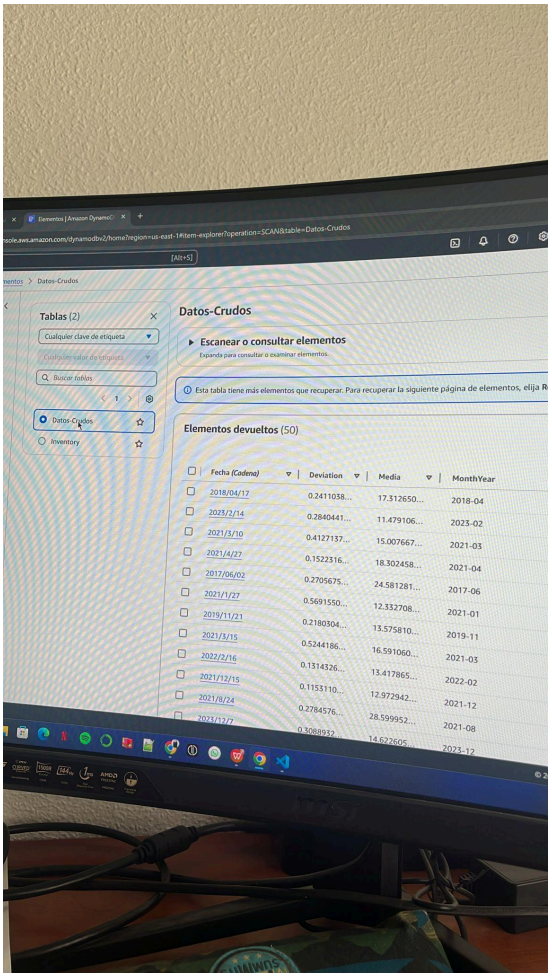
Comprobación de la llegada de correos tras la carga de archivos .csv. (Pd: Al crear la función primeramente pudimos obtener esta captura, pero al tener en nuestra contra el grave problema de la inhabilitación de las cuentas AWS no hemos podido obtener más capturas acerca de la mejora que metimos en el código mi compañero Fernando y yo, en la que solamente nos llegaba un correo electrónico con todas las anomalías pertenecientes a los datos subidos.)

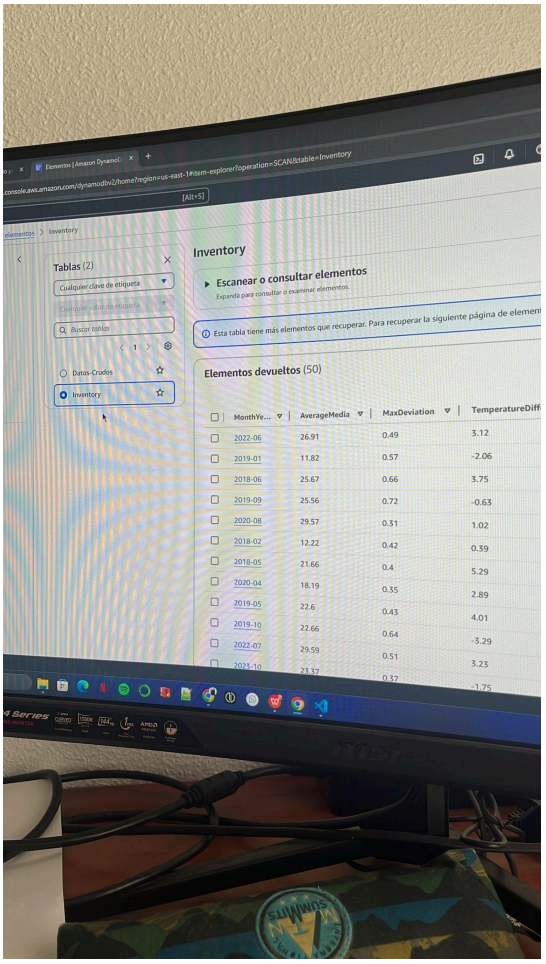


Al bucket le añadimos una notificación de eventos de la primera funcion lambda (funcion-datos), esto nos permite configurar acciones automáticas cuando ocurra un evento específico en el bucket de Amazon S3, como la subida de un archivo, su eliminación, o su replicación. Estas notificaciones son útiles para integrar S3 con otros servicios de AWS y automatizar flujos de trabajo. Lo que nos ayuda a la hora de la asignación de datos a las tablas ya que al subir el archivo en el bucket se realizan las funciones Lambda automáticamente.

Subida de archivos al bucket. (En la fotografía del inicio se observan los datos bien cargados)

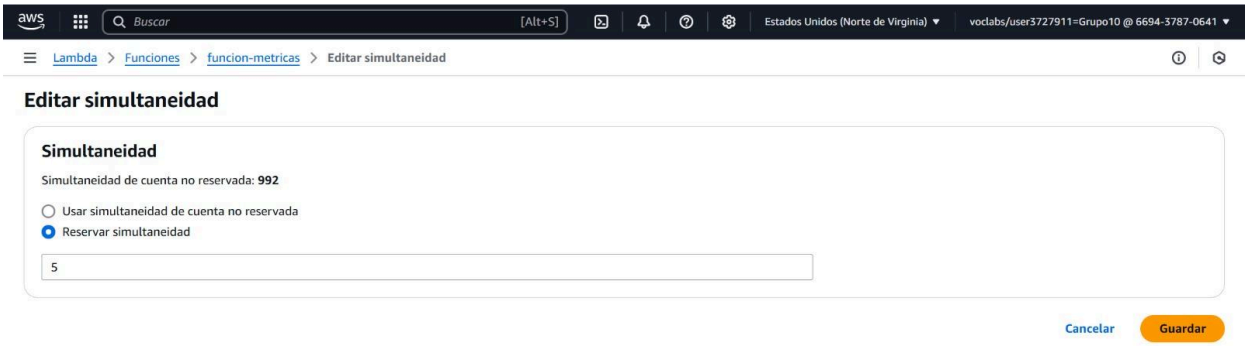


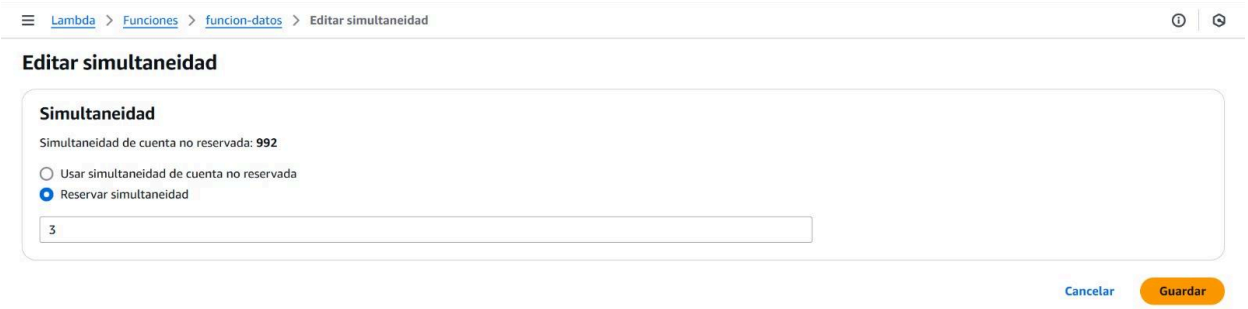




Estas fotografías fueron realizadas en una de las realizaciones de prueba del proyecto antes de la desactivación de las cuentas. Pero creemos que pueden servir para que observe que realmente nos funcionaba.

Para solucionar el error de la inhabilitación le asignamos un valor específico de simultaneidad a cada función lambda, para limitar el número de veces que se puedan ejecutar las funciones y funcionen separadas y aisladas para no competir entre ellas por dichos recursos.





También realizamos el código de la plantilla CloudFormation para la automatización en la creación de la base de datos, está adjuntado en la entrega pero le explicamos un poco acerca del código:

Este código define una infraestructura automatizada en AWS CloudFormation que utiliza varios servicios, incluidos DynamoDB, Lambda, SNS y S3, para procesar y analizar datos, generar métricas, y enviar notificaciones automáticas basadas en umbrales definidos. A continuación, se detalla cada sección del código:

1. Tablas DynamoDB

Tabla Datos-Crudos

● Esta tabla almacena los datos en bruto provenientes de archivos CSV subidos a S3.
● Esquema de Clave Primaria:
○ Hash Key: Fecha (tipo: S).
● Índice Secundario Global (GSI):
○ Nombre: MonthYear-index.
○ Clave de índice: MonthYear (tipo: S).
○ Proyección: ALL (todas las columnas).
● Especificación de Flujo DynamoDB: NEW_AND_OLD_IMAGES, lo que permite activar eventos en función de cambios en los datos.

Tabla Datos-Metricas

● Almacena métricas agregadas mensuales calculadas a partir de los datos en la tabla Datos-Crudos.
● Esquema de Clave Primaria:
○ Hash Key: MonthYear (tipo: S).

2. Función Lambda: funcion-tabla-datos

Esta función procesa archivos CSV subidos a S3 y almacena los datos en la tabla Datos-Crudos.

Flujo de Trabajo:

1. Activación: La función se activa mediante eventos de S3 cuando un archivo es subido.

2. Descarga del Archivo: Descarga el archivo CSV al almacenamiento temporal de Lambda (/tmp).

3. Procesamiento del CSV:
○ Lee el archivo fila por fila.
○ Calcula el campo MonthYear basado en la columna Fecha.○ Valida y transforma datos como Media y Deviation.

4. Almacenamiento en DynamoDB: Inserta cada fila procesada como un ítem en la tabla Datos-Crudos mediante un escritor por lotes (batch_writer).

Propósito:
Esta función automatiza el procesamiento inicial de datos, permitiendo que el proyecto cargue grandes volúmenes de información en bruto en DynamoDB para análisis posterior.

3. Función Lambda: funcion-tabla-metricas

Calcula métricas mensuales y las almacena en la tabla Datos-Metricas.
Flujo de Trabajo:

1. Activación: Se activa mediante flujos de DynamoDB al detectar nuevos ítems en la tabla Datos-Crudos.

2. Cálculo de Métricas:
○ Consulta todos los registros de un mes específico (MonthYear) en la tabla Datos-Crudos.
○ Calcula métricas :
■ Promedio de Media.
■ Máxima Desviación.
■ Máxima Temperatura del Mes.
■ Diferencia de Temperatura Máxima con el Mes Anterior.

3. Almacenamiento en DynamoDB: Inserta o actualiza las métricas calculadas en la tabla Datos-Metricas.

Propósito:
Proporciona un análisis agregado de los datos mensuales, permitiendo la generación de reportes o el monitoreo de tendencias.

4. Función Lambda: funcion-notificacion

Envía alertas por correo electrónico a través de SNS si las desviaciones de temperatura superan un umbral definido (0.5).

Flujo de Trabajo:

1. Activación: Se activa mediante flujos de DynamoDB al detectar cambios en la tabla Datos-Crudos.

2. Evaluación de Umbral:
○ Revisa el campo Deviation de los registros nuevos o actualizados.○ Si el valor excede 0.5, genera un mensaje de alerta con detalles como la
fecha y el valor de desviación.

3. Publicación de la Alerta:
○ Publica el mensaje en el tema SNS configurado (TemperatureAlert).
○ Los suscriptores del tema (como una dirección de correo electrónico) reciben la notificación.

Propósito:
Permite la detección temprana de anomalías en los datos y notifica automáticamente al equipo encargado, mejorando la capacidad de respuesta.

5. Servicios Adicionales

SNS (Simple Notification Service):

● Tema: TemperatureAlert.
○ Actúa como un canal de notificación centralizado para alertas relacionadas con desviaciones de temperatura.
● Suscripción:
○ Dirección de correo electrónico: jlabellan@gmail.com.
○ Permite enviar alertas en tiempo real al destinatario.
Flujos DynamoDB (Streams):
● Habilitados en la tabla Datos-Crudos con el tipo NEW_AND_OLD_IMAGES.
● Estos flujos permiten disparar funciones Lambda (funcion-tabla-metricas y funcion-notificacion) automáticamente al detectar cambios en los datos.

6. Importancia de Esta Infraestructura para el Proyecto

1. Automatización Completa del Flujo de Trabajo:
○ Desde la carga de datos en bruto (archivos CSV) hasta el análisis mensual y la generación de alertas, todo el proceso está automatizado.

2. Escalabilidad y Bajo Costo:
○ DynamoDB con PAY_PER_REQUEST asegura que solo se pague por las operaciones realizadas.
○ Las funciones Lambda son escalables automáticamente según la carga.

3. Notificaciones en Tiempo Real:
○ La integración con SNS permite alertar de inmediato sobre anomalías críticas, mejorando la capacidad de reacción del equipo.

4. Procesamiento Dinámico:
○ Las funciones Lambda manejan eventos de manera asíncrona, optimizando el tiempo de procesamiento sin intervención manual.

5. Seguridad y Resiliencia:
○ Las operaciones están aisladas mediante flujos de trabajo independientes para procesamiento de datos, análisis y alertas.

Tarea 3: Implementación Servicio Web

A continuación, seguimos con la creación de una pila a raíz de una plantilla de CloudFormation con la intención de automatizar gran parte del proceso, en este caso hicimos dos plantillas, la primera es en la que utilizábamos un balanceador de carga para el servicio del Docker y la segunda fue la que usamos utilizando la instancia sin el balanceador, explicaremos el proceso de ambas y mostraremos su correcto funcionamiento.

Explicación plantilla sin balanceador:

Este código define una infraestructura automatizada mediante una plantilla de AWS CloudFormation, la cual despliega recursos en la nube para un proyecto que utiliza servicios RESTful y Docker. A continuación, se detalla cada sección, explicando los recursos creados y su papel dentro del proyecto:

Red Virtual: VPC (Virtual Private Cloud)

La VPC es el núcleo de la red privada del proyecto. Se crea una red virtual privada con el rango de direcciones IP 192.168.0.0/16. Esto permite aislar y gestionar de forma segura los recursos desplegados.

Para habilitar el acceso a Internet, se incluye un Internet Gateway, que actúa como el punto de conexión entre la VPC y el mundo exterior. Este gateway se asocia con la VPC mediante un VPCGatewayAttachment, permitiendo que los recursos internos puedan comunicarse con servicios externos.

Subredes Públicas

Dentro de la VPC, se crean dos subredes públicas con rangos de IP: 192.168.1.0/24 y 192.168.2.0/24. Estas subredes están distribuidas en dos zonas de disponibilidad distintas, lo que mejora la tolerancia a fallos al garantizar que los recursos se mantendrán operativos incluso si una zona experimenta problemas.

Se configura una tabla de rutas pública (PublicRouteTable) que dirige el tráfico de las subredes hacia el Internet Gateway. Cada subred se asocia con esta tabla de rutas mediante asociaciones específicas, lo que asegura que las subredes tengan conectividad con Internet.

Además, se integran reglas básicas de control de tráfico utilizando las asociaciones con los controles de acceso predeterminados de la VPC.

Instancia EC2 con Docker

Se lanza una instancia EC2 llamada "DockerInstance", que actúa como el servidor principal del proyecto. Esta instancia utiliza la AMI ami-08a0d1e16fc3f61ea, seleccionada por su compatibilidad con Docker.

En esta instancia se instala Docker automáticamente al iniciarse, gracias a un script de configuración (UserData). Docker es crucial porque facilita el despliegue y la gestión de contenedores, donde se ejecutará la aplicación Flask que proporciona los servicios RESTful.

Para complementar la instancia, se crea un volumen de almacenamiento EBS de 100 GB que se adjunta como almacenamiento persistente. Este volumen permite que los datos críticos, como bases de datos o registros, se mantengan disponibles incluso si la instancia se reinicia.

Grupos de Seguridad

Los grupos de seguridad definen las reglas de acceso a los recursos:

- FlaskSecurityGroup: Permite el tráfico entrante en el puerto 5000 (donde opera la API Flask) y en el puerto 22 (para acceso remoto mediante SSH). Este grupo protege la instancia EC2 al limitar el tráfico a lo estrictamente necesario.
- BalanceadorSecurityGroup: Permite que las solicitudes HTTP sean recibidas por el balanceador de carga en el puerto 5000.
- TaskSecurityGroup: Controla el tráfico entre el balanceador de carga y las instancias EC2, asegurando que solo el balanceador pueda comunicarse con las aplicaciones.

Estos grupos refuerzan la seguridad de la infraestructura al minimizar posibles accesos no autorizados.

Almacenamiento y Persistencia

Un volumen EBS de 100 GB se configura y se monta en la instancia EC2. Este disco adicional es fundamental para almacenar datos persistentes, lo que resulta vital para un proyecto que maneja grandes volúmenes de información o necesita guardar configuraciones críticas.

Outputs

Al finalizar la creación de los recursos, se exportan los identificadores de la VPC, las subredes y la instancia EC2. Estos datos son útiles para integrarse con otras aplicaciones o plantillas de infraestructura, ya que permiten reutilizar los recursos de manera eficiente.

Importancia de esta Infraestructura para el Proyecto

1. Conectividad y Escalabilidad: La VPC y las subredes públicas ofrecen una infraestructura de red robusta y escalable para desplegar aplicaciones. La segmentación de las subredes mejora la disponibilidad y la gestión de tráfico.
2. Ejecución del Servicio RESTful: La instancia EC2 con Docker proporciona un entorno ágil para desplegar y gestionar la aplicación Flask, que implementa las APIs RESTful necesarias para el proyecto.
3. Seguridad: Los grupos de seguridad garantizan que solo el tráfico autorizado llegue a los recursos, protegiendo la infraestructura de ataques externos.
4. Almacenamiento Persistente: El volumen EBS asegura que los datos importantes del proyecto se almacenen de forma duradera, incluso si la instancia EC2 se reinicia o se actualiza.
5. Automatización: Al utilizar CloudFormation, se logra un despliegue automatizado y reproducible de toda la infraestructura, lo que ahorra tiempo y minimiza errores en comparación con la configuración manual.

Explicación de la pila sin balanceador:

Sección de Recursos (Resources)

Esta es la sección principal donde se define la infraestructura:

1. VPC (Virtual Private Cloud):
    - Crea una VPC con soporte para DNS habilitado.
    - CIDR asignado: 192.168.0.0/16.
    - Se etiqueta con el nombre "proyecto-vpc".
2. Internet Gateway:
    - Se crea un Internet Gateway, que es un componente que permite que la VPC tenga conectividad con Internet.
    - Se asocia a la VPC mediante un VPCGatewayAttachment.
3. Public Route Table:
    - Define una tabla de rutas para la VPC.
    - Se configura una ruta que dirige todo el tráfico de salida (0.0.0.0/0) hacia el Internet Gateway.

4. Public Subnet:
    - Se crea una subred pública en la VPC con el rango de direcciones IP 192.168.1.0/24.
    - Se despliega en una Zona de Disponibilidad seleccionada dinámicamente.
    - Se etiqueta como "public-subnet-1".
5. PublicSubnetRouteTableAssociation:
    - Asocia la subred pública (PublicSubnet) con la tabla de rutas pública.
6. PublicSubnetNetworkAclAssociation:
    - Asocia la subred pública con la ACL predeterminada de la VPC.
7. DockerInstance (EC2 Instance):
    - Despliega una instancia EC2 con las siguientes características:
        - Tipo de instancia: t2.micro.
        - Imagen AMI: ami-08a0d1e16fc3f61ea (preconfigurada para CentOS/RedHat).
        - Par de claves SSH para acceso: dockerInstance-keypair.
        - Asociada a la subred pública.
        - Se le asigna una IP pública.
        - Ejecuta un script de UserData que actualiza el sistema operativo e instala Docker.
    - La instancia está protegida por un grupo de seguridad específico (FlaskSecurityGroup).
8. DiskVolume (EC2 Volume):
    - Crea un volumen EBS de 100 GB en la misma zona de disponibilidad que la instancia EC2.
9. DiskMountPoint (EC2 VolumeAttachment):
    - Adjunta el volumen EBS (DiskVolume) a la instancia EC2 como un disco adicional en /dev/sdh.
10. FlaskSecurityGroup (EC2 Security Group):
    - Define un grupo de seguridad con las siguientes reglas:
        - Permite el tráfico TCP en el puerto 5000 (usualmente para Flask).
        - Permite conexiones SSH (puerto 22) desde cualquier IP.
    - Aplica a la VPC.

Sección de Salidas (Outputs)

Esta sección define las salidas que proporciona la plantilla una vez que la infraestructura se ha desplegado:

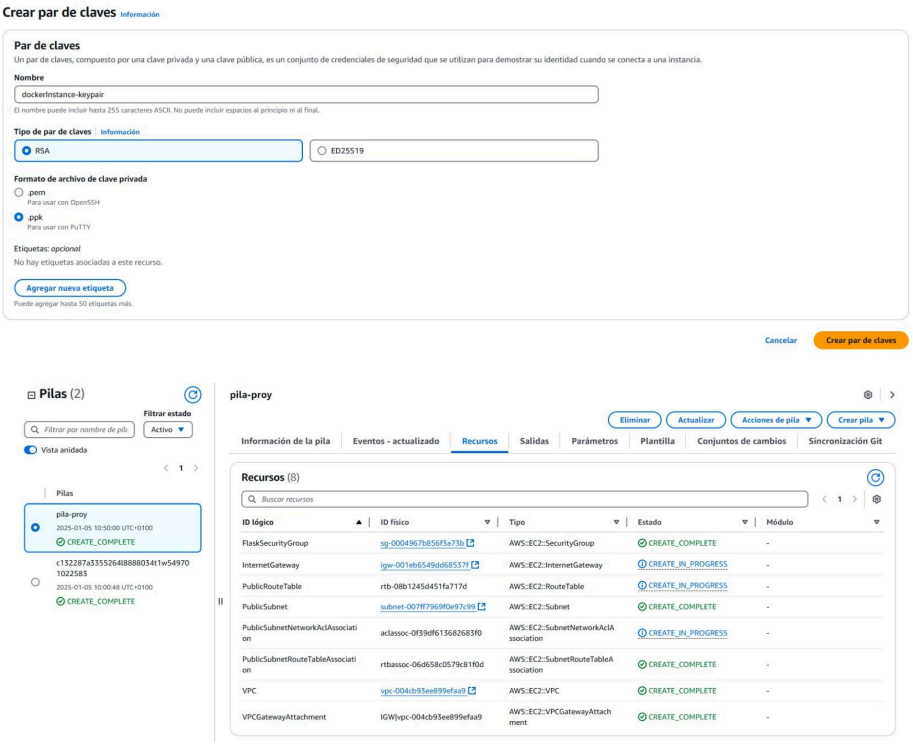
1. PublicSubnet:
    - Devuelve el ID de la subred pública y lo exporta con el nombre del stack.
2. DockerInstance:
    - Devuelve el ID de la instancia EC2 creada.
3. VPC:
    - Devuelve el ID de la VPC y lo exporta con el nombre del stack.

Resumen General

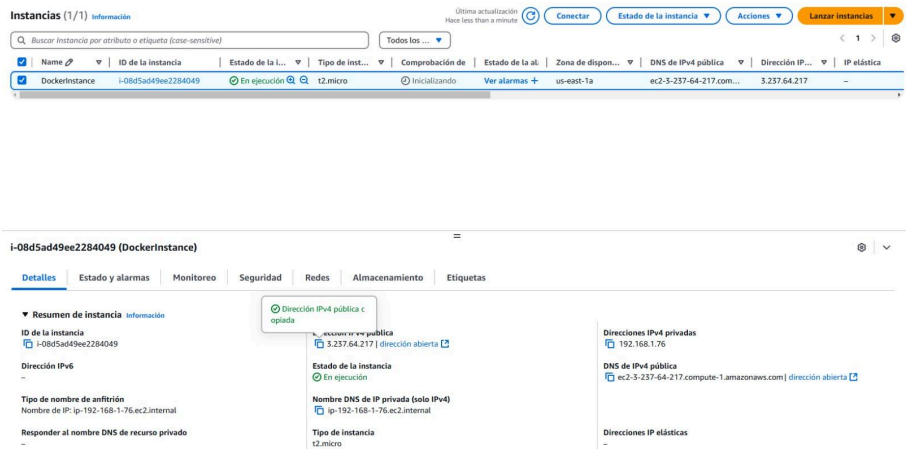
Esta plantilla implementa una infraestructura básica que incluye:

- Una VPC.
- Una subred pública conectada a Internet mediante un Internet Gateway.
- Una instancia EC2 que instala Docker y tiene un volumen de 100 GB adjunto.
- Un grupo de seguridad que permite acceso SSH y tráfico en el puerto 5000.

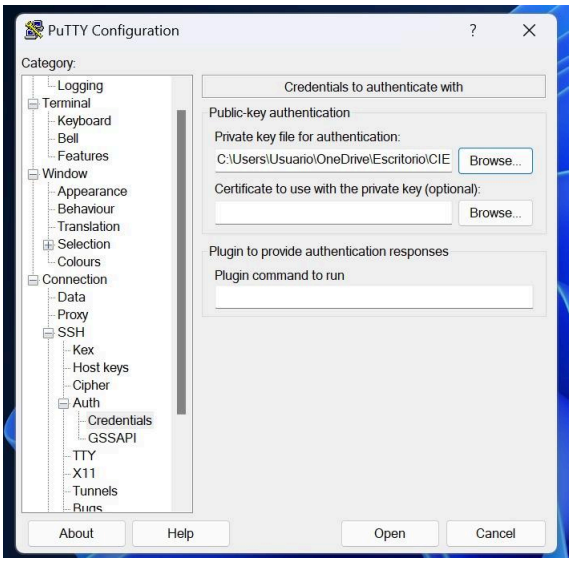
Luego de la creación de la pila (antes de la creación de la pila debemos crear el par de claves) continuamos con el uso de PUTTY para la creación del contenedor docker, primeramente con la ip pública de la instancia Docker creada con la pila, iniciamos el putty:



Ahora vamos a PUTTY:



Con esa IP Pública



Entramos en el terminal y creamos el directorio con mkdir flask_docker, luego creamos los 3 archivos necesarios para el contenedor Docker, la app_flask.py, los requirements.txt y el Dockerfile.

Todo con nano flask_docker/nombre_archivo, y al hacer ls en flask_docker los observamos, una vez ya creados todos los ficheros, expliquemos para que sirve cada uno:

- **App Flask:** Este código es una aplicación web en Flask que expone una API para consultar métricas climáticas almacenadas en una tabla DynamoDB llamada Metricas, utilizando la biblioteca boto3. La API ofrece tres endpoints: /maxdiff, que devuelve la máxima diferencia de temperatura para un mes y año específicos; /sd, que obtiene la desviación estándar máxima para esos mismos parámetros; y /temp, que devuelve la temperatura promedio del período. Los parámetros month y year son obligatorios en las solicitudes, y los datos son consultados en DynamoDB con la clave primaria compuesta MonthYear (formato YYYY-MM). Además, cada respuesta incluye el ID de la instancia EC2 desde donde se

ejecuta la aplicación, obtenido mediante ec2_metadata. La aplicación se ejecuta en el puerto 5000 y permite conexiones desde cualquier host (0.0.0.0).

- **Requirements:** Es un archivo de dependencias (generalmente llamado requirements.txt) que especifica las bibliotecas necesarias para que el proyecto funcione.
- **Dockerfile:** define las instrucciones necesarias para crear una imagen de Docker que contenga y ejecute la aplicación Flask.

Tras esto, debemos inicializar el docker con sudo systemctl start docker y continuar creando la imagen del contenedor con sudo docker build -t flask_container . , tras esto comprobamos las imágenes Docker con sudo docker images y ejecutamos una instancia de la imagen Docker que acabamos de crear mediante sudo docker run -p 5000:5000 -d flask_container, finalmente, comprobamos mediante netstat -tulnp que hay tráfico en el puerto 5000 y nos queda comprobar que el servidor funciona y nos devuelve los datos pedidos.

```
ec2-user@ip-192-168-1-76 ~]$ mkdir flask_docker
ec2-user@ip-192-168-1-76 ~]$ nano flask_docker/app.py
ec2-user@ip-192-168-1-76 ~]$ nano flask_docker/requirements.txt
ec2-user@ip-192-168-1-76 ~]$ nano flask_docker/Dockerfile
```

```
[ec2-user@ip-192-168-1-76 flask_docker]$ ls
Dockerfile  app.py  requirements.txt
[ec2-user@ip-192-168-1-76 flask_docker]$ sudo systemctl start docker
[ec2-user@ip-192-168-1-76 flask_docker]$ sudo docker build -t flask_container .
[+] Building 12.1s (12/12) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                          0.0s
 => => transferring dockerfile: 278B                                          0.0s
 => resolve image config for docker.io/docker/dockerfile:1.4                  0.4s
 => docker-image://docker.io/docker/dockerfile:1.4@sha256:9ba7531bd80fb0a     0.6s
 => => resolve docker.io/docker/dockerfile:1.4@sha256:9ba7531bd80fb0a8586     0.0s
 => => sha256:1328b32c40fca9bcf9d70d8eccb72eb873d1124d72d 9.94MB / 9.94MB     0.3s
 => => sha256:9ba7531bd80fb0a858632727cf7a112fbfd19b17e94 2.00kB / 2.00kB     0.0s
 => => sha256:ad87fb03593d1b71f9a1cfc1406c4aafcb253b1dabebf56 528B / 528B     0.0s
 => => sha256:1e8a16826fd1c80a63fa6917a9c7284c94e40cded14 2.37kB / 2.37kB     0.0s
 => => extracting sha256:1328b32c40fca9bcf9d70d8eccb72eb873d1124d72dadce0     0.2s
 => [internal] load .dockerignore                                             0.0s
 => => transferring context: 2B                                               0.0s
 => [internal] load metadata for docker.io/library/python:3.8-alpine          0.3s
 => [1/5] FROM docker.io/library/python:3.8-alpine@sha256:3d93b1f77efce33     1.6s
 => => resolve docker.io/library/python:3.8-alpine@sha256:3d93b1f77efce33     0.0s
 => => sha256:cd39247b02f92df84605637e0f1b7887434169aca10 4.95kB / 4.95kB     0.0s
 => => sha256:43c4264eed91be63b206e17d93e75256a6097070ce6 3.62MB / 3.62MB     0.1s
 => => sha256:bbd039b2f2dc0cbe5a1fb925d1006dc58892cd7 455.12kB / 455.12kB     0.1s
 => => sha256:2fe9ed713579c200eb3576e25f4604247c6b5d364 14.22MB / 14.22MB     0.4s
 => => sha256:3d93b1f77efce339aa77db726656872517b0d6783 10.29kB / 10.29kB     0.0s
 => => sha256:9f956f4ecf00137962b8da05d2ced2181cf22e23036 1.73kB / 1.73kB     0.0s
 => => extracting sha256:43c4264eed91be63b206e17d93e75256a6097070ce643c5e     0.3s
 => => sha256:86e037ebf8b67184256b8126401150c6d34e6d9977b5f32 247B / 247B     0.2s
 => => extracting sha256:bbd039b2f2dc0cbe5a1fb925d1006dc58892cd76b385af72     0.1s
 => => extracting sha256:2fe9ed713579c200eb3576e25f4604247c6b5d36482a5bb4     0.8s
 => => extracting sha256:86e037ebf8b67184256b8126401150c6d34e6d9977b5f320     0.0s
 => [internal] load build context                                             0.0s
 => => transferring context: 4.07kB                                           0.0s
 => [2/5] WORKDIR /app                                                        0.2s
 => [3/5] COPY requirements.txt requirements.txt                              0.0s
 => [4/5] RUN pip3 install -r requirements.txt                                7.6s
 => [5/5] COPY . .                                                            0.1s
 => exporting to image                                                        1.0s
 => => exporting layers                                                       0.9s
 => => writing image sha256:9a08426560a5466f9baa120709877b51fce04a6b6bb7b     0.0s
 => => naming to docker.io/library/flask_container                            0.0s
[ec2-user@ip-192-168-1-76 flask_docker]$
```

```
[ec2-user@ip-192-168-1-76 flask_docker]$ sudo docker images
REPOSITORY        TAG       IMAGE ID       CREATED              SIZE
flask_container   latest    9a08426560a5   About a minute ago   98MB
[ec2-user@ip-192-168-1-76 flask_docker]$ sudo docker run -p 5000:5000 -d flask_container
ceeab894bc6df25252b333ab16dd1ede625a4cf9bb0b5177978287368fa07e0a
[ec2-user@ip-192-168-1-76 flask_docker]$ sudo docker ps
CONTAINER ID   IMAGE             COMMAND            CREATED          STATUS          PORTS
                    NAMES
ceeab894bc6d   flask_container   "python3 app.py"   16 seconds ago   Up 15 seconds   0.0.0.0:5000->5000/
tcp, :::5000->5000/tcp   awesome_feynman
[ec2-user@ip-192-168-1-76 flask_docker]$
```

No es seguro 100.27.205.71:5000/sd?month=3&year=2017

Dar formato al texto


```
{
  "instancia": "i-0400766b83b2c4d45",
  "max_sd": "0.4",
  "month": "3",
  "year": "2017"
}
```


Como se puede observar el servidor también nos funcionaba en las distintas pruebas realizadas durante la creación del proyecto, por lo que el servidor también funciona correctamente devolviendo los valores pedidos en dicha consulta.

Finalmente, se puede comprobar el correcto funcionamiento del pipeline de datos, cargando y asignando los valores adecuados a cada una de las tablas como enviando el correo electrónico de aviso. Y por último, el correcto funcionamiento del servidor web y la correcta obtención de los datos a través de cada consulta realizada.

Resumen de recursos y servicios desplegados

Crea cuantas filas como sea necesario. Debes incluir todos los recursos que hacen posible que tu despliegue funcione correctamente.

Identificador de recurso	Nombre de recurso	Tipo de recurso en AWS	Comentario (opcional)
	bucket-proyecto-icap-grupo	***bucket***	***Almacenan los archivos csv agregados***
	funcion-datos	**Función lambda**	**Encargada de obtener los datos del bucket y elaborarlos para guardarlos dentro dela tabla Datos-Crudos.**
	funcion-metricas	**Función lambda**	**Encargada de obtener los datos de la tabla Datos-Crudos y elaborarlos para guardarlos dentro de la tabla Metrics (mensualmente).**
	funcion-sns	**Función lambda**	**Encargada de obtener los datos de la tabla Datos-Crudos y observar cuales superan dicho umbral para poder notificarlo.**

	Datos-Crudos	**Tabla Dynamo**	**Almacena los datos semanales y a parte tiene la columna indice de MonthYear**
	Metrics	**Tabla Dynamo**	**Almacena los datos mensuales que son los que queremos obtener en las peticiones http**
	AlarmaTemperatura	**Tema SNS**	**Encargado del sistema de notificación por email**
	dockerInstance-keypair	**Par de claves**	**par de claves necesario la iniciación de putty con la ip de la instancia docker**
	proyecto-VPC	**VPC**	**red virtual aislada dentro de la nube de AWS, donde se puede definir y controlar la topología, subredes, direccionamiento IP, y configuraciones de seguridad, proporcionando un entorno privado para ejecutar recursos como instancias EC2 y bases de datos.**
	public-subnet-1	**SubRed**	**división dentro de la VPC, donde se pueden organizar y aislar recursos permitiendo controlar la comunicación entre dentro de la subred y con el resto de la red o Internet.**

	DockerInstance	**EC2**	**gestiona contenedores que contienen aplicaciones o servicios de forma aislada y pórtatil.**
	FlaskSecurityGroup	**Sec-Group**	**Grupo de seguridad de la instancia Docker que permite tráfico en el puerto 22 y 5000.**

Pasos para la Replicación:

1.- Creación de la base de datos

Primero cree las tablas Dynamo:

- Datos-Crudos: Clave partición Fecha, y crear índice llamado MonthYear
- Metrics: Clave partición MonthYear

2.- Crear Bucket: llamarlo bucket-proyecto-icap-grupo10

3.- Crear funciones Lambda:

funcion-datos: llamarla funcion-datos, tiempo ejecución Python 3.12, rol LabRole. Introducir el código del archivo funcion_datos.py

funcion-metricas: llamarla funcion-metricas, tiempo ejecución Python 3.12, rol LabRole. Introducir el código del archivo funcion_metricas.py, crear desencadenador de Dynamo y buscar tabla Datos-Crudos

funcion-sns: llamarla funcion-sns, tiempo ejecución Python 3.12, rol LabRole. Introducir el código del archivo funcion_sns.py, crear desencadenador de Dynamo y buscar tabla Datos-Crudos

4.- Crear notificacion de evento en el bucket: en propiedades del bucket, crear notificacion de eventos, llamada funcion-datos, Tipos de evento: Todos los eventos de creación de objetos, destino función Lambda, seleccionamos la función lambda funcion-datos

5.- Crear tema SNS: En Simple Notification Service, crear tema, ingresar AlarmaTemperatura y en detalles Estándar. Creamos el Tema, luego creamos la suscripción, le damos a crear suscripción y elegimos protocolo correo electrónico, punto de enlace escogemos el correo del profesor y creamos la suscripción.

6.- Agregamos los archivos csv si queremos comprobar que la parte de la base de datos funciona, y así tenemos ya los datos para poder realizar peticiones web.

7.- Creación servicio web, en el servicio EC2 creamos un par de claves llamado dockerInstance-keypair, luego en el servicio Cloudformation creamos la pila con la plantilla que le hemos subido llamada: proyecto-iac-servicioweb-tenza-pinera.yaml. Tras esto, se crearán los recursos. Ahora, debemos ir a la DockerInstance y copiar su IP pública.

8.- Imagen Docker: Ahora en PUTTY pegamos la ip publica, ponemos un tiempo de 30ms, y desplegamos Auth para meter el par de claves. Ponemos ec2-user y accedemos. Luego tenemos que poner estos códigos:

mkdir flask_docker

nano flask_docker/app_flask.py -> Introducimos el código de app_flask.py subido.

nano flask_docker/requirements.txt -> Introducimos el contenido de requirements.txt

nano flask_docker/Dockerfile -> Introducimos el contenido de Dockerfile.txt

cd flask_docker

ls

sudo systemctl start docker

sudo docker build -t flask_container .

sudo docker images

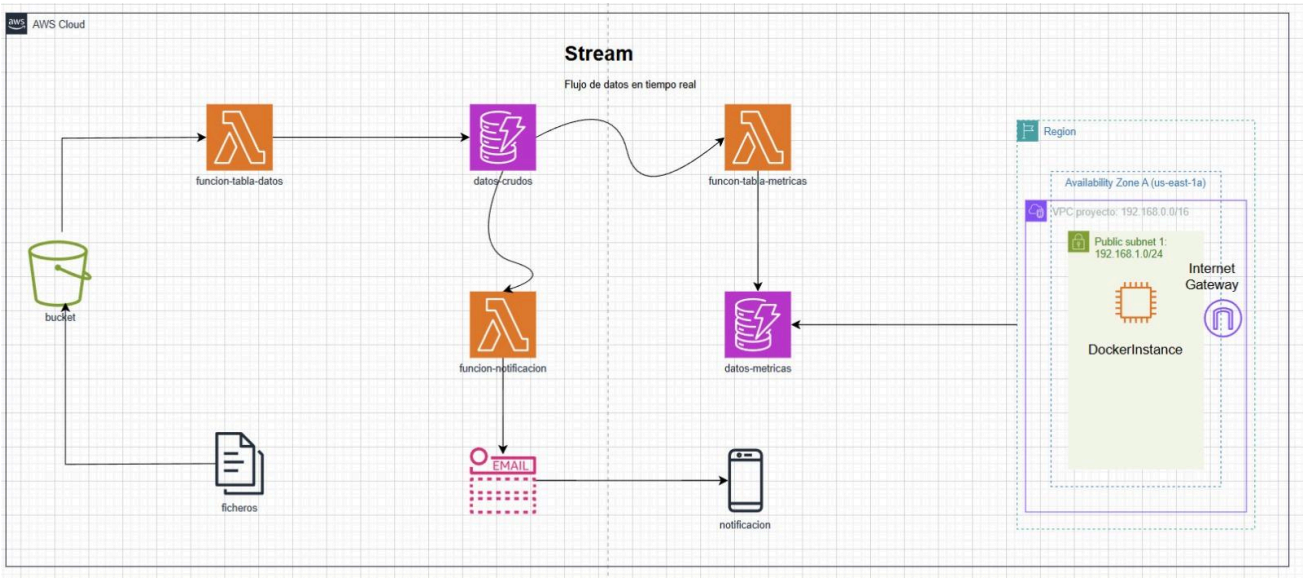
sudo docker run -p 5000:5000 -d flask_container

sudo docker ps

netstat -tulnp

Y ya tendremos la instancia de la imagen Docker corriendo, vamos al navegador y realizamos la peticion: http://[IP-address] /maxdiff?month=3&year =2017

Por último mostrar el diagrama creado con los recursos principales:



En el diagrama podemos encontrar los recursos principales de nuestro proyecto, como se puede observar, los ficheros son subidos al bucket, donde se almacenan, y pasan a una función lambda, es decir la de datos crudos para insertarlos en la tabla dynamodb.

Luego de esto, la tabla datos crudos sirve como desencadenador para las posteriores funciones lambda, la que almacena los datos mensuales con los valores de las métricas y la que notifica el servicio al superar la desviación. Se puede ver el tema de SNS con la notificación y la asignación de dichos datos mensuales a la otra tabla dynamo db.

Tras esto en la parte derecha se observa la vpc, con la subred pública y la instancia Docker, con el internet gateway y todo dentro de la zona de disponibilidad, esto es creado por la pila de CloudFormation y nos ayuda a poder crear el servicio web final desde Putty con la inicialización de la imagen Docker, el servicio web está conectado en el diagrama a la tabla de los datos mensuales ya que son estos valores los que se mostrarán cuando el cliente realice una petición web.

Anexo de plantillas CloudFormation:

1.- Plantilla de Automatización en la creación de la base de datos.

```
AWSTemplateFormatVersion: "2010-09-09"
```

```
Resources:

  # Bucket S3

  S3BucketProyecto:

    Type: AWS::S3::Bucket

    Properties:

      BucketName: deposito-proyecto



  # Tabla DynamoDB Datos-Crudos

  DatosCrudosTable:

    Type: AWS::DynamoDB::Table

    Properties:

      TableName: Datos-Crudos

      AttributeDefinitions:

        - AttributeName: Fecha

          AttributeType: S

        - AttributeName: MonthYear

          AttributeType: S

      KeySchema:

        - AttributeName: Fecha

          KeyType: HASH

      GlobalSecondaryIndexes:

        - IndexName: MonthYear-index
```

```
        KeySchema:
          - AttributeName: MonthYear
            KeyType: HASH
        Projection:
          ProjectionType: ALL
    BillingMode: PAY_PER_REQUEST
    StreamSpecification:
      StreamViewType: NEW_AND_OLD_IMAGES

# Tabla DynamoDB Datos-Metricas
DatosMetricas:
  Type: AWS::DynamoDB::Table
  Properties:
    TableName: Metrics
    AttributeDefinitions:
      - AttributeName: MonthYear
        AttributeType: S
    KeySchema:
      - AttributeName: MonthYear
        KeyType: HASH
    BillingMode: PAY_PER_REQUEST

# Función Lambda funcion-A
FuncionTablaDatos:
  Type: AWS::Lambda::Function
```

```
    Properties:
      FunctionName: funcion-datos
      Runtime: python3.12
      Role: arn:aws:iam::669437870641:role/LabRole
      Handler: index.lambda_handler
      Code:
        ZipFile: |
        import boto3
        import json
        import urllib.parse
        import csv
        from decimal import Decimal
        from datetime import datetime

        s3 = boto3.resource('s3')
        dynamodb = boto3.resource('dynamodb')

        table_raw = dynamodb.Table('Datos-Crudos')

        def lambda_handler(event, context):
            # Registramos el evento que activa la función Lambda
                    print("Evento recibido por la función Lambda: " +
json.dumps(event, indent=2))

            # Extraemos el nombre del bucket y la clave del archivo desde el
evento
```

```
            bucket = event['Records'][0]['s3']['bucket']['name']

                                                                    key      =
urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])

             local_filename = '/tmp/data.csv'  # Especificamos una ubicación
temporal para el archivo


            # Intentamos descargar el archivo CSV desde el bucket de S3

            try:

                s3.meta.client.download_file(bucket, key, local_filename)

                print(f'Archivo {key} descargado desde el bucket {bucket}')

            except Exception as e:

                print(f'Error al descargar el archivo: {e}')

                 raise e  # Detenemos la ejecución si no podemos descargar el
archivo


                 # Leemos y procesamos el archivo CSV para cargar datos en
DynamoDB

            try:

                with open(local_filename, encoding='utf-8') as csvfile:

                     reader = csv.DictReader(csvfile)  # Leemos cada fila del
archivo como un diccionario

                    with table_raw.batch_writer() as batch:

                        for row in reader:

                            try:

                                # Registramos la fila que estamos procesando

                                print(f"Procesamos la fila: {row['Fecha']}")
```

```
                                        # Transformamos y preparamos los datos
necesarios

                                        date = datetime.strptime(row['Fecha'],
'%Y/%m/%d')

                                    month_year = date.strftime('%Y-%m')

                                    media = Decimal(row['Medias'])

                                    deviation = Decimal(row['Desviaciones'])


                                      # Construimos el ítem que vamos a cargar en
DynamoDB

                                    raw_item = {

                                        'Fecha': row['Fecha'],

                                        'MonthYear': month_year,

                                        'Media': media,

                                        'Deviation': deviation,

                                    }


                                    # Insertamos el ítem en la tabla de DynamoDB

                                    batch.put_item(Item=raw_item)

                                    print(f"Ítem añadido: {raw_item}")


                                except Exception as e:

                                     # Registramos cualquier error al procesar una
fila y continuamos

                                                print(f"Error al procesar la fila
{row['Fecha']}: {e}")

                                    continue
```

```
            print("¡Datos cargados exitosamente en DynamoDB!")
        except Exception as e:
            # Registramos cualquier error al procesar el archivo CSV
            print(f"Error al procesar el archivo CSV: {e}")
            raise e




# Función Lambda funcion-B
FuncionTablaMetricas:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: funcion-tabla-metricas
    Runtime: python3.12
    Role: arn:aws:iam::669437870641:role/LabRole
    Handler: index.lambda_handler
    Code:
      ZipFile: |
        import boto3
        import json
        from datetime import datetime, timedelta
        from decimal import Decimal
        from boto3.dynamodb.conditions import Key
```

```
          # Conectamos con los recursos de DynamoDB

          dynamodb = boto3.resource('dynamodb')

            table_raw = dynamodb.Table('Datos-Crudos')  # Tabla que contiene
los datos crudos

           table_metrics = dynamodb.Table('Metrics')  # Tabla que almacenará
las métricas calculadas



          def lambda_handler(event, context):

              # Registramos el evento recibido por la función Lambda

                      print("Evento  recibido  por  la  función  Lambda:  "  +
json.dumps(event, indent=2))


                 # Utilizamos un `batch_writer` para realizar escrituras por
lotes en DynamoDB

              with table_metrics.batch_writer() as batch:

                     processed_month_years = set()  # Conjunto para evitar
procesar duplicados del mismo mes-año


                  try:

                      # Iteramos sobre los registros del evento

                      for record in event['Records']:

                          newImage = record['dynamodb'].get('NewImage', None)
# Verificamos si existe una nueva imagen

                         if newImage:

                             # Extraemos los datos del registro

                             new_item = record['dynamodb']['NewImage']

                                 month_year = new_item['MonthYear']['S']  #
Obtenemos el mes-año del registro
```

```
                                    # Saltamos el procesamiento si ya hemos
manejado este mes-año
                                if month_year in processed_month_years:
                                    continue


                                    processed_month_years.add(month_year)   #
Marcamos este mes-año como procesado


                                # Consultamos los datos del mes-año en la tabla
`Datos-Crudos`
                                response = table_raw.query(
                                        IndexName='MonthYear-index',   # Índice
secundario que permite buscar por MonthYear

KeyConditionExpression=Key('MonthYear').eq(month_year)
                                )
                                  monthly_data = response['Items']  # Obtenemos
los ítems de la consulta


                                # Calculamos las métricas del mes actual
                                   total_media = sum(Decimal(item['Media']) for
item in monthly_data)  # Suma total de las medias
                                  max_deviation = max(Decimal(item['Deviation'])
for item in monthly_data)  # Máxima desviación
                                   max_temperature = max(Decimal(item['Media'])
for item in monthly_data)  # Máxima temperatura
                                   record_count = len(monthly_data)  # Contamos
los registros del mes
```

```
                                    average_media = total_media / record_count if
record_count > 0 else 0  # Calculamos el promedio


                                # Calculamos el mes-año anterior

                                    current_date = datetime.strptime(month_year +
'-01', '%Y-%m-%d')  # Convertimos el mes-año a fecha

                                                        previous_month_year  =
(current_date.replace(day=1)  -  timedelta(days=1)).strftime('%Y-%m')    #  Mes
anterior


                                # Consultamos los datos del mes-año anterior

                                previous_response = table_raw.query(

                                    IndexName='MonthYear-index',


KeyConditionExpression=Key('MonthYear').eq(previous_month_year)

                                )

                                                        previous_month_data  =
previous_response['Items']


                                    # Calculamos la temperatura máxima del mes
anterior (si existen datos)

                                max_temperature_previous = max(

                                            Decimal(item['Media'])  for  item  in
previous_month_data

                                ) if previous_month_data else None


                                 # Calculamos la diferencia de temperatura entre
el mes actual y el anterior
```

```
                                temperature_difference = (
                                    max_temperature - max_temperature_previous
                                    if max_temperature_previous is not None
                                    else None
                                )

                                # Construimos el ítem para la tabla `Metrics`
                                metrics_item = {
                                    'MonthYear': month_year,
                                     'AverageMedia': round(average_media, 2),  #
Promedio de las medias
                                     'MaxDeviation': round(max_deviation, 2),  #
Máxima desviación
                                                                 'TemperatureDifference':
round(temperature_difference, 2) if temperature_difference is not None else
None,  # Diferencia de temperatura
                                }

                                # Insertamos el ítem en la tabla `Metrics`
                                batch.put_item(Item=metrics_item)
                                        print(f'Datos mensuales actualizados en
Metrics: {metrics_item}')

                    except Exception as e:
                        # Registramos el error si ocurre algún problema durante
el procesamiento
                       print(e)
```

```
                            print("Error al procesar el evento del trigger de
DynamoDB.")
                        raise e




  # Desencadenador de DynamoDB Streams para Lambda B
  DynamoDBStreamLambdaPermission:
    Type: AWS::Lambda::Permission
    Properties:
      FunctionName: !Ref FuncionTablaMetricas
      Action: lambda:InvokeFunction
      Principal: dynamodb.amazonaws.com
      SourceArn: !GetAtt DatosCrudosTable.StreamArn


  LambdaBEventSourceMapping:
    Type: AWS::Lambda::EventSourceMapping
    Properties:
      EventSourceArn: !GetAtt DatosCrudosTable.StreamArn
      FunctionName: !Ref FuncionTablaMetricas
      StartingPosition: LATEST


  # Tema SNS
  TemperatureAlertTopic:
    Type: AWS::SNS::Topic
    Properties:
```

```
    TopicName: TemperatureAlert


# Suscripción al tema SNS
TemperatureAlertSubscription:
  Type: AWS::SNS::Subscription
  Properties:
    Protocol: email
    Endpoint: pineratenza@gmail.com
    TopicArn: !Ref TemperatureAlertTopic


# Función Lambda funcion-notificacion
FuncionNotificacion:
  Type: AWS::Lambda::Function
  Properties:
    FunctionName: funcion-notificacion
    Runtime: python3.12
    Role: arn:aws:iam::669437870641:role/LabRole
    Handler: index.lambda_handler
    Environment:
      Variables:
        SNS_TOPIC_ARN: !Ref TemperatureAlertTopic
    Code:
      ZipFile: |
        import json
        import boto3
```

```
import os


def lambda_handler(event, context):
    # Registramos el evento recibido por la función Lambda
        print("Evento recibido por la función Lambda:",
json.dumps(event, indent=2))


    # Inicializamos el cliente de SNS
    sns = boto3.client('sns')
    sns_topic_arn = os.environ['SNS_TOPIC_ARN']


    # Creamos una lista para acumular las alertas encontradas
    alerts = []


    try:
        # Iteramos sobre los registros recibidos en el evento
        for record in event['Records']:
            # Extraemos la nueva imagen del registro
            new_image = record['dynamodb'].get('NewImage', None)


            # Verificamos si la nueva imagen contiene los campos
'Deviation' y 'Fecha'
            if new_image and 'Deviation' in new_image and 'Fecha'
in new_image:
                try:
```

```
                                    # Convertimos la desviación a tipo float y
extraemos la fecha
                                deviation = float(new_image['Deviation']['N'])
                                fecha = new_image['Fecha']['S']


                                # Verificamos si la desviación excede el umbral
de 0.5
                                if deviation > 0.5:
                                    # Agregamos una alerta a la lista
                                    alerts.append(f"Fecha: {fecha}, Desviación:
{deviation}")


                            except ValueError:
                                # Registramos un error si no podemos convertir
la desviación a float
                                print("Error: No se pudo convertir el valor de
'Deviation' a float.")
                        else:
                            # Registramos una advertencia si no se encuentran
los campos necesarios
                            print("Advertencia: No se encontró 'Deviation' o
'Fecha' en la nueva imagen.")


                    # Si encontramos alertas, construimos y enviamos un mensaje
consolidado
                    if alerts:
                        message = (
                            "¡Alerta de Temperatura!\n\n"
```

```
                                "Se han detectado desviaciones semanales que
exceden el umbral de 0.5:\n\n"
                            + "\n".join(alerts)
                        )

                        # Registramos el mensaje que vamos a enviar
                        print("Enviando alerta consolidada:", message)

                        # Publicamos el mensaje en el tópico de SNS
                        sns.publish(
                            TopicArn=sns_topic_arn,
                            Message=message,
                            Subject='Resumen de Alertas de Temperatura',
                        )

                        # Devolvemos una respuesta con el número de registros
procesados y alertas encontradas
                    return {
                        'statusCode': 200,
                                        'body': f'Se procesaron exitosamente
{len(event["Records"])} registros. '
                                f'Se encontraron {len(alerts)} desviaciones que
exceden el umbral.'
                    }

                except Exception as e:
```

```
                        # Registramos cualquier error que ocurra durante el
procesamiento
                    print(f"Error al procesar los registros: {str(e)}")
                    return {
                        'statusCode': 500,
                        'body': f"Error al procesar el evento: {str(e)}"
                    }



  # Permiso para que DynamoDB invoque la función Lambda
  DynamoDBStreamLambdaPermissionNotification:
    Type: AWS::Lambda::Permission
    Properties:
      FunctionName: !Ref FuncionNotificacion
      Action: lambda:InvokeFunction
      Principal: dynamodb.amazonaws.com
      SourceArn: !GetAtt DatosCrudosTable.StreamArn


  # Mapeo de fuente de eventos para conectar DynamoDB Streams con la función
Lambda
  LambdaNotificationEventSourceMapping:
    Type: AWS::Lambda::EventSourceMapping
    Properties:
      EventSourceArn: !GetAtt DatosCrudosTable.StreamArn
      FunctionName: !Ref FuncionNotificacion
      StartingPosition: LATEST
```

2.- Plantilla de Automatización en la creación de los recursos para el servicio web:

```
AWSTemplateFormatVersion: 2010-09-09

Description: >-

  Network Template: Sample template that creates a VPC with 2 public subnets
and a docker instance EC2.




######################

# Resources section

######################


Resources:


  ## VPC


  VPC:

    Type: AWS::EC2::VPC

    Properties:

      EnableDnsSupport: true

      EnableDnsHostnames: true
```

```
    CidrBlock: 192.168.0.0/16
    Tags:
      - Key: Name
        Value: "proyecto-vpc"

## Internet Gateway

InternetGateway:
  Type: AWS::EC2::InternetGateway

VPCGatewayAttachment:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    VpcId: !Ref VPC
    InternetGatewayId: !Ref InternetGateway

## Public Route Table
PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC

PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: VPCGatewayAttachment
```

```
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

## Public Subnet

PublicSubnet:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 192.168.1.0/24
    # MapPublicIpOnLaunch: true
    AvailabilityZone: !Select
      - 0
      - !GetAZs
        Ref: AWS::Region
    Tags:
      - Key: Name
        Value: "public-subnet-1"

# PublicSubnet2:
#   Type: AWS::EC2::Subnet
#   Properties:
#     VpcId: !Ref VPC
```

```
#     CidrBlock: 192.168.2.0/24
#     # MapPublicIpOnLaunch: true
#     AvailabilityZone: !Select
#       - 1
#       - !GetAZs
#         Ref: AWS::Region
#     Tags:
#       - Key: Name
#         Value: "public-subnet-2"

PublicSubnetRouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet
    RouteTableId: !Ref PublicRouteTable

# PublicSubnetRouteTableAssociation2:
#   Type: AWS::EC2::SubnetRouteTableAssociation
#   Properties:
#     SubnetId: !Ref PublicSubnet2
#     RouteTableId: !Ref PublicRouteTable

PublicSubnetNetworkAclAssociation:
  Type: AWS::EC2::SubnetNetworkAclAssociation
  Properties:
```

```
    SubnetId: !Ref PublicSubnet
    NetworkAclId: !GetAtt
      - VPC
      - DefaultNetworkAcl


# PublicSubnetNetworkAclAssociation2:
#   Type: AWS::EC2::SubnetNetworkAclAssociation
#   Properties:
#     SubnetId: !Ref PublicSubnet2
#     NetworkAclId: !GetAtt
#       - VPC
#       - DefaultNetworkAcl


# ## Private subnets
# PrivateSubnet1:
#   Type: AWS::EC2::Subnet
#   Properties:
#     VpcId: !Ref VPC
#     CidrBlock: 192.168.3.0/24
#     AvailabilityZone: !Select
#       - 0
#       - !GetAZs
#         Ref: AWS::Region
#     Tags:
#       - Key: Name
```

```
#         Value: privatesubnet1


# PrivateSubnet2:
#   Type: AWS::EC2::Subnet
#   Properties:
#     VpcId: !Ref VPC
#     CidrBlock: 192.168.4.0/24
#     AvailabilityZone: !Select
#       - 1
#       - !GetAZs
#         Ref: AWS::Region
#     Tags:
#       - Key: Name
#         Value: privatesubnet2


# NatGatewayIP:
#   Type: AWS::EC2::EIP
#   DependsOn: VPCGatewayAttachment
#   Properties:
#     Domain: vpc


# NatGateway:
#   Type: AWS::EC2::NatGateway
#   Properties:
#     AllocationId: !GetAtt NatGatewayIP.AllocationId
```

```
#     SubnetId: !Ref PublicSubnet


# PrivateRouteTable1:
#   Type: AWS::EC2::RouteTable
#   Properties:
#     VpcId: !Ref VPC
#     Tags:
#       - Key: Name
#         Value: privateroutetable1


# DefaultPrivateRoute1:
#   Type: AWS::EC2::Route
#   Properties:
#     RouteTableId: !Ref PrivateRouteTable1
#     DestinationCidrBlock: 0.0.0.0/0
#     NatGatewayId: !Ref NatGateway


# PrivateSubnet1RouteTableAssociation:
#   Type: AWS::EC2::SubnetRouteTableAssociation
#   Properties:
#     RouteTableId: !Ref PrivateRouteTable1
#     SubnetId: !Ref PrivateSubnet1


# PrivateRouteTable2:
#   Type: AWS::EC2::RouteTable
```

```
#   Properties:
#     VpcId: !Ref VPC
#     Tags:
#       - Key: Name
#         Value: private-route-table2

# DefaultPrivateRoute2:
#   Type: AWS::EC2::Route
#   Properties:
#     RouteTableId: !Ref PrivateRouteTable2
#     DestinationCidrBlock: 0.0.0.0/0
#     NatGatewayId: !Ref NatGateway

# PrivateSubnet2RouteTableAssociation:
#   Type: AWS::EC2::SubnetRouteTableAssociation
#   Properties:
#     RouteTableId: !Ref PrivateRouteTable2
#     SubnetId: !Ref PrivateSubnet2

DockerInstance:
  Type: AWS::EC2::Instance

  Properties:
    InstanceType: t2.micro
    ImageId: ami-08a0d1e16fc3f61ea
```

```
      KeyName: dockerInstance-keypair
      IamInstanceProfile: LabInstanceProfile
      NetworkInterfaces:
        - GroupSet:
            - !Ref FlaskSecurityGroup
          AssociatePublicIpAddress: true
          DeviceIndex: 0
          DeleteOnTermination: true
          SubnetId: !Ref PublicSubnet
      Tags:
        - Key: Name
          Value: DockerInstance
      UserData:
        Fn::Base64: !Sub |
          #!/bin/bash
          dnf update -y
          #install Docker
          dnf -y install docker


  DiskVolume:
    Type: AWS::EC2::Volume
    Properties:
      Size: 100
      AvailabilityZone: !GetAtt DockerInstance.AvailabilityZone
    DeletionPolicy: Snapshot
```

```
DiskMountPoint:
  Type: AWS::EC2::VolumeAttachment
  Properties:
    InstanceId: !Ref DockerInstance
    VolumeId: !Ref DiskVolume
    Device: /dev/sdh

FlaskSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Enable TCP ingress for Flask
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 5000
        ToPort: 5000
        CidrIp: 0.0.0.0/0
      - IpProtocol: tcp
        FromPort: 22
        ToPort: 22
        CidrIp: 0.0.0.0/0
    Tags:
      - Key: Name
        Value: Flask Security Group
```

```
# BalanceadorSecurityGroup:
#   Type: AWS::EC2::SecurityGroup
#   Properties:
#     GroupDescription: Enable TCP ingress for Flask
#     VpcId: !Ref VPC
#     SecurityGroupIngress:
#       - IpProtocol: tcp
#         FromPort: 5000
#         ToPort: 5000
#         CidrIp: 0.0.0.0/0
#     Tags:
#       - Key: Name
#         Value: BalanceadorSecurityGroup



# TaskSecurityGroup:
#   Type: AWS::EC2::SecurityGroup
#   DependsOn: BalanceadorSecurityGroup
#   Properties:
#     GroupDescription: Enable traffic from ALB SG
#     VpcId: !Ref VPC
#     SecurityGroupIngress:
#       - IpProtocol: tcp
#         FromPort: 5000
```

```
  #         ToPort: 5000
  #         SourceSecurityGroupId: !Ref BalanceadorSecurityGroup
  #     Tags:
  #       - Key: Name
  #         Value: TaskSecurityGroup




#####################
# Outputs section
#####################


Outputs:


  PublicSubnet:
    Description: The subnet ID to use for public web servers 1
    Value: !Ref PublicSubnet
    Export:
      Name: !Sub '${AWS::StackName}-Subnet1ID'


  PublicSubnet2:
    Description: The subnet ID to use for public web servers 2
    Value: !Ref PublicSubnet2
    Export:
      Name: !Sub '${AWS::StackName}-Subnet2ID'
```

```
DockerInstance:
  Description: EC2 instance ID
  Value: !Ref DockerInstance


VPC:
  Description: VPC ID
  Value: !Ref VPC
  Export:
    Name: !Sub '${AWS::StackName}-VPCID'
```